

MALICIOUS URL DETECTION USING MACHINE LEARNING

Group Number: 19

Group Members

Aarya Balwadkar
Afifa Bintul Hasan
Aparna Nair
Eccha Bansal

*Department of Computer Science / Computer Engineering
Symbiosis Institute of Technology, Pune | Academic Year 2026–27*

Abstract—The widespread use of web services has made the Uniform Resource Locator (URL) one of the main vectors of phishing attacks, malware distribution, spamming, and other forms of cyberfraud. While blacklist-based defense mechanisms continue to work for known dangers, their effectiveness is limited by the fact that they are built on previous observations of the URL or domain and therefore cannot defend against new URLs or those that have been modified. This case study studies machine learning techniques to detect malicious URLs, with emphasis on features that can be extracted from the URL directly. The publicly available dataset ISCX-URL2016 consisting of multiclass URLs such as benign, spam, phishing, malware, and defacement [4] is chosen for the present research work. The methodology presented in this paper involves acquisition of datasets, data cleansing, exploratory analysis, URL normalization, feature extraction, feature selection, stratification of the dataset, and evaluation using performance measures like accuracy, precision, recall, F1 score, and ROC-AUC. No practical implementation is performed and no results are produced for this research. Rather, it presents a methodological framework along with a literature review of the possible role of machine learning models in enhancing existing URL reputation systems.

Index Terms—Malicious URL Detection; Phishing; Machine Learning; Lexical Features; Feature Engineering; Cybersecurity; Random Forest; XGBoost; URL Classification; Web Security.

I. INTRODUCTION

A. Background

The Uniform Resource Locator (URL) represents the established way to locate resources on the World Wide Web. The structure of a URL generally consists of a scheme such as `http://` or `https://`, a hostname/domain name, a possible subdomain, a path, and a query, port number, and fragment, wherever necessary. In light of the fact that URLs help users in accessing web content, such an identical technique enables attackers to guide their victims to malicious infrastructure.

Malicious URLs refer to web pages that take users to content whose purpose is to cause harm. The types of malicious URLs in the literature vary based on the type of harm they cause. They include:

Phishing URLs – Web pages that appear to come from an established company in order to acquire login details, payment data, or other sensitive information.

Malware URLs – Links that will make the user's device download malware such as worms, trojan horses, viruses, ransomware, and spyware.

Spam URLs – Links sent out in unsolicited mass mailings and that promote worthless or even fraudulent products and services.

Defacement URLs are hyperlinks leading to pages where there has been an unauthorized alteration of the content, done usually in order to convey a certain ideological message or to demonstrate a successful breach.

Scam URLs are pages designed in line with social engineering stories, such as fake lottery or fake technical support scams.

B. URL Obfuscation and the Difficulty of Detection

Malicious links are always found together with a certain level of obfuscation. Some of the common obfuscation methods include the use of IP addresses instead of domain names, the use of '@' to make the preceding string be ignored in the host name resolution process, the addition of many subdomains to suggest that the URL is connected to an already familiar brand like `secure-login.paypal.example.com`, the use of hexadecimal and percent encoding to obscure characters, or using a URL shortener to disguise the link.

C. Limitations of Traditional Blacklist and Whitelist Approaches

Blacklist continues to be the most widely used technique, that is, maintaining a list of URLs or domains that have been found to be malicious based on user reports, honeypots and vendor intelligence feeds; tools like Google Safe Browsing use such a mechanism. Whitelist involves using the opposite technique whereby only certain domains are allowed access. While blacklists may be easy to implement, cheap to look up and reliable, the literature highlights several flaws:

Blacklisting occurs reactively. A URL will have to be seen, reported, and confirmed before being added to a blacklist, and thus, there is a period of time where it could be used against targets without hindrance.

Attackers can register many temporary domain names or make small changes to a blacklisted URL, which will allow them to evade fixed entries through relatively cheap means.

Blacklists do not generalize; a URL that has never been seen before is never evaluated, even when the structure matches one already flagged as being malicious.

D. The Role of Machine Learning in URL Classification

Machine Learning approaches this problem as a classification problem in general terms, instead of performing a straightforward look up. Instead of checking whether the particular URL has been seen before, the machine learning algorithm will recognize certain patterns which help identify the malicious URL against the non-malicious one based on features like abnormal length, unusual distribution of characters, unusual depth of subdomains or suspicious keywords. This way, machine learning algorithms will be able to classify a URL for the first time it comes across, which makes machine learning ideal for zero day URLs. In modern security applications, both methods are often combined, where the blacklist is used for a quick initial analysis, and the model is used for handling unknown cases. In case of classification based solely on lexical features of the URL string, there is no need for retrieving, rendering, or executing the target, which saves time and increases safety while decreasing reliance on signatures.

E. Relevance of the Topic in Today's World

Malicious URL detection is the point of intersection of two trends: the dependency of many everyday processes on web applications, and the implementation of machine learning in cybersecurity. In order to separate facts from conclusions, this paragraph will first provide industry statistics that have been confirmed, then the findings of the literature review, and finally the analysis of the project team.

1) *Established Facts — Scale of the Threat:* According to reports from the Anti-Phishing Working Group (APWG), an industry body which has been tracking phishing since 2003, there were 3.8 million unique phishing websites in 2025, compared to 3.76 million in 2024, out of which there were 853,244 unique websites registered during Q4 2025. The quarterly reports from APWG also reveal that Software as a Service (SaaS), webmail, and social media still constitute the top spoofed industries. Also, there was an increase of 57% in the incidence of business email compromise frauds involving wire transfer in Q3 2025, with an average request amount of USD 48,115. Independent of that, the Internet Crime Complaint Center (IC3) of the United States Federal Bureau of Investigation reported in 2024 that there were 193,407 complaints regarding phishing and spoofing incidents, along with reported losses arising out of business email compromise amounting to USD 2.77 billion. Based on their own official publications, these statistics confirm that URL-based criminal activity remains widespread and, in many aspects, is growing.

2) *Findings Reported by Researchers:* In fact, there is an abundance of literature on this topic, reviewed in section 5 of this report, that proves that it is possible to achieve good classification results using the characteristics of the URL alone. The classification task itself was defined in the work of Sahoo, Liu, and Hoi and divided into feature groups used in further research. The studies carried out between 2022 and 2025 have shown yet again that ensemble tree algorithms, such as Random Forest and XGBoost, often produce performance

comparable to deep learning solutions in the case of tabular URL features with significantly fewer computing resources used. This is one of the reasons behind the ongoing interest in classical solutions. At the same time, the same research has also revealed growing worries about the issue of concept drift and the creation of URLs specifically aimed at bypassing the classifier.

3) *Our Interpretation:* Industry-level data combined with the research results show that the detection of malicious URLs is not just a theory but also a practical problem in the sphere of cybersecurity. The increased use of internet banking, e-commerce, telecommuting, and cloud computing increases the risk of interaction with unknown links accessed through e-mail, instant messengers, QR codes, and social networking sites, according to APWG publications. A system that can evaluate a new URL instantly, operate under the limitations of a browser/mail gateway environment, and can be updated economically as the attacks change is needed to solve an actual problem. The team finds the subject applicable in practice because the subject employs a fast learning pipeline to counteract a dynamically changing threat but still based on the fundamentals of machine learning and web security.

F. Problem Statement

Because of the inefficiencies associated with static black and white-listing filters discussed in Section 1, an approach is needed that will be able to differentiate between malicious and non-malicious URLs based on the features intrinsic to the URL, making evaluation of new URLs possible in a timely manner. Hence, the key research question of this case study is whether machine learning models can distinguish between malicious and legitimate URLs by analyzing their intrinsic features.

The scope of the claims must be clearly defined. This system, which is discussed here, is not claimed to be a universal fix for cybersecurity problems, neither is it supposed to take the place of other layers of security like blacklisting systems, browser isolation, email authentication protocols (SPF, DKIM, DMARC), user awareness programs, or endpoint security systems. Instead, this system serves as a complementary layer for addressing the issue that list-based methods face with the problem of dealing with new and unknown malicious URLs.

G. Objectives

The purpose of this research is to meet the following objectives:

To conduct a comprehensive literature review to examine the state-of-the-art approaches for representing malicious URLs using lexical, host-based, and hybrid features. This objective would help in identifying the sets of features and classifiers which have performed well based on the research findings.

To design a list of lexical and structural characteristics of a URL that can be used to differentiate between malicious and legitimate URLs without downloading and executing the destination page.

To design a preprocessing pipeline for a malicious-URL data set.

In order to extract a short set of relevant and non-redundant features from the URLs that have been identified using lexical analysis and suspicious indicators.

In order to train multiple classical classifiers, which include Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbours, and XGBoost among others.

In order to test and compare the classifiers on the basis of their Accuracy, Precision, Recall, F1-score, and ROC-AUC values with emphasis on false-negative values because of the security aspect involved.

To gauge the usefulness, limitations, and implementation of the lexical features based URL detection method, and suggest directions for future research.

II. LITERATURE REVIEW

This subsection reviews 15 papers related to malicious URL and phishing detection in the peer-reviewed as well as pre-print literature; these papers range from the year 2009 up to 2025 as a time span when survey articles are written. The analysis is done by category of feature extraction approach: lexical, hybrid, hostname, and deep representation features. Details of all referenced articles can be found in References (Section 21) as per IEEE number.

A. Foundational and Survey Work

Ma et al. (1998) are some of the pioneers who approached the task of malicious websites classification from the perspective of statistical learning. They proved that linguistic and host properties of URLs, which could be collected without actually visiting the target site, could be combined with online machine learning to detect suspicious URLs. This has led to an evolution of the field towards moving away from blacklisting towards more sophisticated solutions.

However, Sahoo et al. [2] proposed a review which characterizes the field by four categories of features: lexical, host-based, content-based, and others while differentiating traditional machine learning from representation learning methods. It is mentioned in the paper that lexical features are computationally cheap and do not need to interact with the destination server. However, models that use only lexical features could be less robust against URL obfuscation attacks in comparison with models using host/content-based information.

An article from 2025 [3] takes this classification to include modern representation methods, such as Transformer models and graph neural networks, organizing them by the modality of data: lexical strings, HTML, JavaScript behavior, and visual representations. The authors highlight that classical learning based on lexical features is a good low-latency benchmark in the practical studies, even after the advances in deep methods.

B. Lexical-Feature-Based Classical Machine Learning Studies

In the paper by Mamun, Rathore, Lashkari, Stakhanova, and Ghorbani [4], ISCX-URL2016 dataset was introduced

and proved to be enough to distinguish between benign and phishing, malware, spam, and defacement URLs using the set of lexical characteristics in combination with the Random Forest classifier. Since its introduction, the dataset has been popular benchmarking dataset.

Joshi, Lloyd, Westin, and Seethapathy [5] developed a 1,023 dimensional representation of URLs using 23 lexical features engineered manually and 1,000 Word2Vec type of features based on trigrams. Using one feature per correlated pair (with correlation coefficient greater than 0.75), the authors have trained Random Forest and reported high discriminative power using only URL information without adding host and content data.

Vanhoenshoven, Napoles, Falcon, Vanhoof, and Koppen [6] carried out the comparison of classical classifiers including Naive Bayes, k-Nearest Neighbours, and tree-based ensemble methods using only lexical URL features. The authors found that the tree-based ensemble methods perform better than probabilistic ones on their dataset, which was repeated by most of other works that use only lexical features and are cited in this paper.

Sahingoz, Buber, Demir, and Diri [7] combined word-level natural language features with seven classical classification algorithms and developed a large dataset called "Big URL Data Set" that includes 36,400 benign URLs and 37,175 phishing URLs. Random Forest with natural language and word vector features reported 97.98% accuracy, and this paper is widely cited in the field of phishing URL classification using only URL text.

Aljabri, Altamimi, and others [8] compared lexical, network, and content feature sets both in classical and deep learning methods. It was shown that the combination of all types of features performs better than individual feature set, and lexical features provide the highest discriminative power per computational cost.

C. Feature Engineering and Domain/Host-Level Studies

Kara, Ok, and Ozaday [9] were interested in such characteristics as the length of domain name, its composition, and the use of unusual top-level domains. In their evaluation with classifiers, these features showed good discrimination of phishing and benign URLs, thus stressing the importance of careful domain name feature engineering.

Gualberto et al. [10] suggested enhancing traditional lexical features with features extracted via topic modeling. The authors believe that an unsupervised extraction of topics from the URL tokens will complement traditional engineering and improve detection accuracy.

Innab et al. [11] studied the ensemble of machine learning algorithms in terms of phishing detection. Their findings are in line with the general tendency found in this review, i.e., ensembles and boosted-trees algorithms work better than single classifiers with tabular URL features.

D. Hybrid and Deep-Representation Studies

Li et al. [12] presented a stacker ensemble that integrates URL-based and HTML-based features. The inclusion of lim-

ited page content data boosted the performance of lexical-based methods, but the page retrieval adds delay and poses security risks.

Somesha, Pais, Rao, and Rathour [13] compared the performances of different deep structures used for detecting phishing sites with those of classic baseline methods. The performance of deep structures matched or surpassed the performance of baseline methods when there was sufficient data, while classical ensembles remained very competitive and considerably easier to train and deploy.

Le et al. [14] developed URLNet, an end-to-end system that generates URL representation using a convolutional neural network directly from the sequence of characters and words, and without any manual feature extraction. The extracted representation is claimed to have captured certain sequential and semantic structures missed by manually crafted features, at the cost of additional data and computation resources. Similar findings for malicious URLs, file paths, and registry keys generated from raw character input can be found in the works of Saxe and Berlin [15].

According to Opara et al. [16], the combination of raw character strings from the URL together with HTML-derived features proved to be beneficial in phishing page classification compared to a baseline approach that relied solely on URLs. This observation further shows how hybrid content-aware methods benefit from increased accuracy at the expense of additional latency and risk.

E. Synthesis and Relevance to This Case Study

Three key insights generated from the analysis of the fifteen articles discussed above are particularly relevant for the development of the proposed methodology. First, the lexical characteristics based only on the URL provide strong discriminatory capacity with low computation complexity and can thus form the basis of a real-time system; additional gains in performance are possible when either the host or the content is considered, although such gains are relatively small. Second, ensemble tree-based classifiers, including Random Forest and gradient boosted trees such as XGBoost, perform similarly to classical methods and, in some comparisons, even compete with much more complicated deep learning models on tabular URL data. Third, poor generalization of the results between different data sets as well as sensitivity to concept drift and adversarial URLs constitute limitations that have been recurrently noted, which implies that the achieved accuracy on a certain data set should not necessarily generalize to another time period/source.

III. METHODOLOGY

Methodology uses the process pipeline in a step-by-step manner following the procedure discussed in Section 5. Every step has been clearly stated with its purpose and placement in the sequence, since the order is of importance for the validity of the process. For example, parameters for scaling, selection, and resampling should be estimated solely on the basis of training data, as discussed in Section 8.

A. Overview of the Pipeline

The process is a combination of twelve phases: (1) data gathering, (2) data cleaning, (3) exploratory analysis, (4) pre-processing and normalization of URLs, (5) features extraction, (6) features selection, (7) splitting into training, validation, and testing sets, (8) models selection, (9) models training, (10) predictions on held-out data, (11) evaluation by a variety of metrics, and (12) comparison of results from different models.

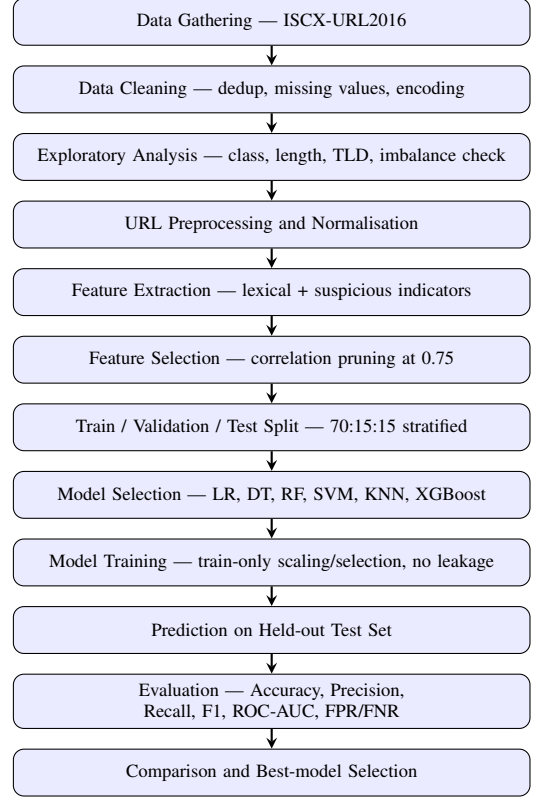


Fig. 1. Methodology pipeline: twelve-phase flowchart from data gathering to model comparison. Scaling, selection and resampling parameters are estimated on training data only.

1) *Data Cleaning*: URLs collected through blacklists, crawl logs, and user reports of phishing URLs often include duplicates, incomplete or invalid entries, unlabelled records, and non-standardized encodings. This is the first preprocessing stage whereby duplicate entries are removed, invalid or incomplete entries are omitted or replaced, and encoding is standardized to make sure that features extraction is done on clean data. Data cleansing comes before exploratory analysis to avoid contamination of summary statistics.

2) *Exploratory Data Analysis (EDA)*: Exploratory analysis describes the distribution of classes, URL length distributions, top level domain frequencies, and any possible artifacts such as clustering of malicious URLs in a short time span or by one source that might cause the model to learn artifact properties instead of true malicious properties. Current results guide the selection of features (Section 9) and determine if there is an imbalance issue (Section 8).

TABLE I
TABLE 1: COMPARATIVE SUMMARY OF THE LITERATURE REVIEWED ON MALICIOUS URL DETECTION

Ref.	Year	Dataset	Feature Type(s)	Model(s)	Key Finding	Limitation Noted
[1]	2009	Proprietary crawled URLs	Lexical + host-based	Online learning (classical)	Established feasibility of statistical URL classification without crawling destination content	Proprietary, non-public dataset limits reproducibility
[2]	2017/19	Multiple (survey)	Lexical, host-based, content-based	Survey of classical + deep methods	Formalised feature taxonomy still used by later work	Survey — no new experimental results
[3]	2025	Multiple (survey)	Lexical, HTML, JS, visual, graph	Survey incl. Transformer/GNN	Classical lexical-ML remains a strong, low-latency baseline vs. deep methods	Newer modalities (visual/GNN) add cost and complexity
[4]	2016	ISCX-URL2016	Lexical	Random Forest and others	Established a widely reused public multi-class URL benchmark	Dataset reflects only 2016-era URL patterns (potential drift)
[5]	2019	Custom crawled corpus	Lexical + tri-gram/Word2Vec	Random Forest	Purely lexical + n-gram features gave strong discrimination	High-dimensional trigram features increase training cost
[6]	2016	Custom dataset	Lexical	Naive Bayes, k-NN, ensembles	Ensemble tree methods outperformed simpler classifiers	Small-scale evaluation; limited generalisability discussed
[7]	2019	Big URL Data Set (73,575 URLs, self-released)	NLP word-vector + lexical	7 classifiers incl. Random Forest	97.98% accuracy reported with NLP + Random Forest	Single-source dataset; possible sampling bias
[8]	2022	Multiple public datasets	Lexical, network, content	Classical ML + deep learning	Combining feature categories improved accuracy over single category	Content/network features require fetching destination site
[9]	2022	URL/domain feature dataset	Domain & lexical features	Classical ML classifiers	Domain-name-level lexical features reliably separated phishing from legitimate sites	Focuses narrowly on domain features; excludes path/query signals
[10]	2020	Custom phishing corpus	Lexical + topic-model features	Classical ML	Topic-modelling features added incremental accuracy gains	Added preprocessing complexity and interpretability cost
[11]	2024	Public phishing dataset	Lexical/derived features	Ensemble ML algorithms	Ensemble methods outperformed single-classifier baselines	Evaluated on a single dataset; generalisation untested
[12]	2019	Custom URL+HTML corpus	Lexical + HTML	Stacking ensemble	Adding HTML features improved on URL-only accuracy	Requires fetching page content, adding latency and risk
[13]	2020	Public phishing datasets	Lexical/derived	Deep learning vs. classical ML	Classical ensembles remained competitive with deep models	Deep models needed larger data volumes to match ensembles
[14]	2018	Public + proprietary URL corpora	Raw character/word sequence (learned)	CNN (URLNet)	Learned representations captured patterns manual features missed	Higher computational cost; less interpretable than lexical features
[16]	2024	Public phishing corpus	Raw URL + HTML (learned)	Deep learning	Hybrid raw-URL + HTML model improved over URL-only baseline	Content fetch dependency; higher pipeline complexity

3) *URL Preprocessing and Normalisation*: Before any numerical representation of the URLs is carried out, the URL string is broken down into its various parts—namely, scheme, host, subdomain, path, query, and fragment—with the help of a common parser library. There is an overall normalization process that involves the conversion of scheme and host components to lowercase, although the path and query strings remain in their original case form since there could be some valuable information within the case difference.

4) *Feature Extraction*: The normalized URLs are then represented using fixed-size vectors, utilizing lexical features and suspicious-indicator features described in Section 9. The process converts the classification task of varying string lengths into supervised learning on a fixed-dimensional feature set, making it possible to use classic classifiers directly.

5) *Feature Selection*: However, the extracted features are characterized by varying degrees of significance, and various length-related metrics demonstrate high inter-correlation levels, as noted in the literature overview provided in Section 5.2. Therefore, by using methods like correlation pruning, feature importance from a preliminary tree construction, or variance-based thresholds, redundancy is decreased, training time is cut down, and overfitting to correlated groups is avoided.

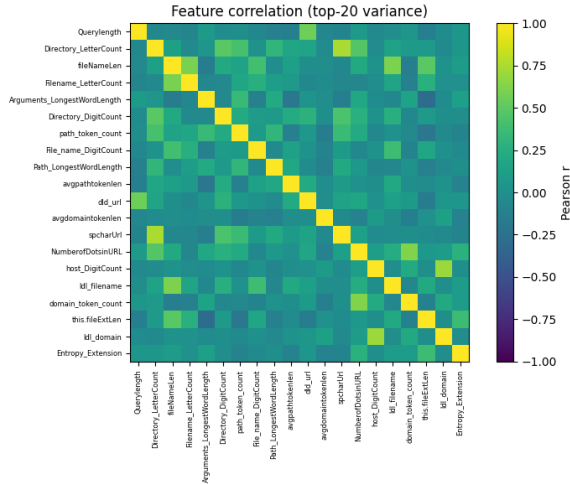


Fig. 2. Figure 1: Pearson correlation among the top-20 highest-variance lexical features on the training set, motivating correlation pruning at 0.75 (original figure).

6) *Train/Validation/Test Splitting*: The cleaned and encoded data set is divided into training, validation, and test subsets in a 70:15:15 ratio, falling in the range of the commonly quoted 70:15:15 to 80:10:10 ratio. The stratified sampling method based on class labels ensures that the proportion of benign to malicious samples remains the same in all subsets. All scaling factors, selection of features, and resampling procedures are learned using only the training subset and then used without any modification for the other subsets (see Section 8.7).

7) *Model Training, Prediction, Evaluation and Comparison*: All models described in Section 10 are trained separately on the same training subset and the same set of features in

order to make sure that any difference we observe is due to the particular algorithm itself rather than the particular dataset used. Optimization of hyperparameters is performed through grid search or random search with cross-validation on the validation subset, whereas all unbiased metrics are calculated based only on the test subset that was not seen neither during training nor during hyperparameter optimization.

B. Dataset Description

Dataset used: ISCX-URL2016 dataset created by Canadian Institute for Cybersecurity at the University of New Brunswick. This dataset contains around 114,400 URLs which belong to categories like Benign, Spam, Phishing, Malware, Defacement, etc., and is described by Mamun et al. [4].

1) *Recommended Candidate Datasets*: ISCX-URL2016 is a multi-class corpus consisting of benign, phishing, malware, spam, and defacement URLs, with precomputed lexical attributes attached. It was introduced by Mamun et al. [4] and became popular among the works surveyed in Section 5.

Big URL Data Set by Sahingoz et al. [7] includes 36,400 legitimate URLs and 37,175 phishing URLs, gathered explicitly for learning-based phishing studies.

PhiUSIIL and GramBeddings are large public datasets used in 2025 for benchmarking phishing detection methods, suitable when a more voluminous and up-to-date sample is necessary to counteract the problem of concept drift.

Additionally, community-sourced Kaggle datasets with malice provenance (i.e., references to such lists as PhishTank, URLHaus, or OpenPhish for malicious URLs) and corresponding legitimate URLs from the domain ranking list can be considered.

Whatever data source is selected, the following documentation should always be supplied: the number of samples, their class definitions and counts, the ratio of benign to malicious samples, the sources and timeframe of collecting the corpus, what features were provided in the dataset and what were computed from strings, and quality issues found by the authors or the authors themselves in the exploratory analysis phase (Section 6.1.3).

2) *Selected Dataset*: The ISCX-URL2016 dataset, which is provided by the Canadian Institute for Cybersecurity at the University of New Brunswick, is adopted to conduct this research. As per the description given, there are five classes defined, namely, Benign, Spam, Phishing, Malware, and Defacement. The benign samples are collected from top-ranked Alexa websites, followed by their filtration via VirusTotal. On the other hand, the malicious samples are gathered from resources such as WEBSPAM-UK2007, OpenPhish, and DNS-BH [4]. The chosen dataset is suitable for the current research since the underlying research examines lightweight lexical analysis.

3) *Class Distribution and Imbalance*: There are about 114,400 URLs in the dataset of ISCX-URL2016. More than 35,300 benign URLs, 12,000 spam URLs, 10,000 phishing URLs, more than 11,500 URLs relating to malware, and more than 45,450 defacement URLs have been reported in the

ISCX-URL2016 dataset [4]. The data is provided as estimates or as underestimates, not exact numbers. This makes multi-class classification and binary classification feasible.

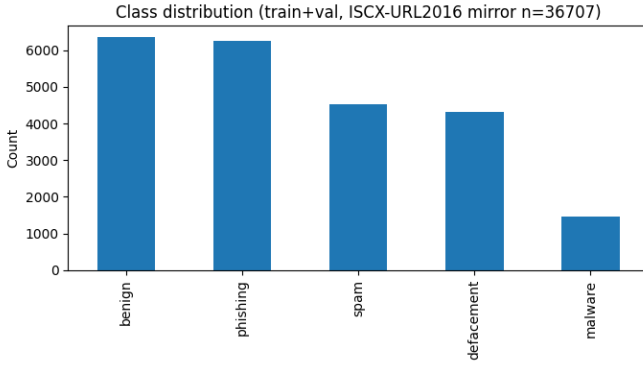


Fig. 3. Figure 2: Class distribution after duplicate removal (n=26,953; defacement majority retained; original figure from project data).

C. Data Preprocessing

Preprocessing makes the raw labeled dataset ready for reliable feature calculation and learning. Each step described below has been done for a particular purpose.

1) *Duplicate Removal*: Exact as well as nearly duplicate addresses, which differ from each other just by a trailing slash, tracking variables, or case sensitivity of the scheme part, are recognized and deleted. It would not be desirable to keep such duplicates in cases where the same address will occur in both the training set and the test set, as the accuracy would then depend on memorization rather than generalization.

2) *Missing-Value Handling*: Entries without a proper label are completely stripped away, as no training nor testing can take place without ground truth. In cases where additional columns are missing, for instance when no WHOIS data is available for a host-based property, a well-established rule is adopted – whether it is the inclusion of an ‘unknown’ category or omission of the target attribute – and it is recorded.

3) *URL Normalisation*: Normalization ensures that trivial format changes do not result in different feature vectors for the same address. The scheme and hostname are converted into lowercase, and path/query remains unchanged. Additionally, percent-encoded symbols are decoded in order to calculate features.

4) *Handling Malformed URLs*: Unparsable addresses due to missing schemes, invalid characters, or truncation are marked, corrected by a rudimentary approach like adding the ‘http://’ scheme, or excluded from the data set. The number of exclusions is kept track of for clear sample size accounting.

5) *Class Imbalance Handling*: If imbalance exists between benign and malicious class samples is identified via exploratory analysis (see Section 6.1.3), techniques such as random undersampling of the majority class, oversampling of the minority class, or creation via SMOTE and similar approaches are used. Otherwise, the classifier could achieve high apparent accuracy through classification of the majority

class alone, achieving low accuracy when it comes to the minority class, typically the malicious one with the highest security implications, see Section 12.

6) *Label Encoding*: The textual classes (‘benign’, ‘phishing’, ‘malware’, and so on) are transformed into numbers for the algorithm to work on them, which is either in the form of binary coding (malicious = 1, benign = 0) in the case of the two-class model or otherwise.

7) *Train/Test Splitting and Avoiding Data Leakage*: As noted in Section 6.1.7, partitioning into training, validation and test sets precedes estimation of any scaling, selection or resampling parameters. Estimating those quantities from the complete collection before splitting would permit information from the test portion to influence training, producing optimistic test figures that would not be reproduced on genuinely new addresses in operational use.

D. Feature Engineering / Feature Extraction

There is a clear distinction between lexical attributes, obtainable from the URL string alone (without network access), and those requiring WHOIS, DNS or similar queries, or knowledge of age of domain, that is, host- or external-based attributes. This distinction, acknowledged in nearly all the studies in Section 5, is important operationally: lexical attributes can be calculated immediately and safely for any address, including one never previously visited, while host-based attributes involve delay, external dependency and, if destination retrieval is required, an element of exposure that a screening tool should preferably avoid.

1) *Lexical Features*: The following lexical attributes are computed directly from the URL string:

Total URL length — overall character count of the complete address.

Hostname length — character count of the domain or host component.

Path length — character count of the path component.

Query length — character count of the query component.

Count of dots (‘.’) — frequently raised in addresses employing multiple subdomains to imitate a trusted brand.

Count of hyphens (‘-’) — commonly present in look-alike or brand-imitating domains (for example, paypal-secure-login.example.com).

Count of slashes (‘/’) — reflective of path depth or redirection sequences.

Count of special symbols (such as ‘?’, ‘=’, ‘&’, ‘%’ and ‘@’).

Count of digits and digit-to-character ratio — malicious addresses at times embed numeric strings to resemble legitimate tracking or identifier parameters.

Subdomain count — number of dot-separated labels preceding the registrable domain.

Shannon entropy of the URL string — an indicator of character-level randomness, often higher in automatically generated malicious domains.

2) *Suspicious Indicator Features*: The following binary or count-based indicators capture structural patterns observed disproportionately in malicious URLs in the reviewed studies:

Use of an IP address in place of a domain name (for example, `http://192.168.1.1/login`).

Occurrence of the '@' symbol, which causes browsers to ignore the part before when resolving hosts..

Too many (like more than 3 or 4) subdomains in the address.

Occurrence of top-level domains that are rare but often found in phishing campaigns.

Usage of shortening domains (like `bit.ly` or `tinyurl.com`), which hide the real destination from the user.

Occurrence of hexadecimal codes or percent-encoded parts in the URL, which are used to encode and hide some parts of text.

Suspicious words in the path or query part like 'login', 'verify', 'secure', 'update' or 'account', while the domain in the beginning does not match.

Usage of a non-standard port in the address. Suspicious host names, like a name that contains mostly randomly generated alphanumeric characters. Want a second opinion?

3) *Host-Based Features (Where the Dataset Supports Them)*: Where the chosen dataset supplies them, host-based attributes may complement the lexical set and, consistent with the studies in Section 5.2, have been reported to provide a further, generally modest, gain over lexical attributes alone:

Domain age: malicious domains tend to be newly-registered while established legitimate domains are less likely to be used for phishing purposes.

WHOIS registration information: country of the domain's registrant, registrar, and whether the registrant's information is hidden.

DNS resolution information: number of resolved addresses, time-to-live values, and name-server diversity.

SSL/TLS certificate information: issuing authority, validity period, and certificate age.

The domain or address reputation, as reported by third-party threat-intelligence sources.

This study considers lexical attributes as the required primary set, since they satisfy the objective of a rapid crawl-free classification expressed in Section 4. Host-based attributes are considered strictly supplementary and are to be included only in case the selected dataset already supplies verified values in order not to introduce unverified or constructed host-level data into the experiment.

E. Machine Learning Models

Six classifiers are proposed, representing linear, tree-based, distance-based and boosted-ensemble principles. The comparison of distinct inductive approaches on a shared feature set is more informative than comparing minor variants within a single family. No superiority is assumed in this section; comparative judgement is left to the results and comparative discussion once experimental evidence is available.

1) *Logistic Regression*: Logistic Regression expresses the log-odds of maliciousness as a linear function of the input attributes, converted to a probability by the sigmoid function. It is a reference point for its speed and interpretability: the sign and magnitude of each coefficient shows whether the corresponding attribute favours a 'malicious' or 'benign' outcome. Its limitation is that it represents only linear boundaries in feature space, which may underfit where the relation between lexical attributes and malicious intent is strongly non-linear — a pattern frequently reported in Section 5. Training and prediction costs are minimal, which is suitable for high-throughput screening.

2) *Decision Tree*: A Decision Tree partitions the feature space according to successive threshold tests (for example, 'URL length > 75?'), chosen at each node to maximise class separation, as measured by Gini impurity or information gain. Such trees are easily interpreted, since the learned splits can be read as plain rules, and express non-linear interactions. Its main drawback is overfitting, resulting in deep and highly specific structures that generalise poorly to unseen addresses, unless depth or leaf size is controlled.

3) *Random Forest*: Random Forest builds a large collection of Decision Trees, each fitted to a bootstrapped sample and a random feature subset, and the label is decided by majority vote. The averaging in this way significantly reduces the overfitting of an individual tree, while retaining the ability to model non-linearities. The method is consistently reported in Section 5 as a reliable baseline for lexical URL classification and additionally provides a feature-importance ranking that is useful for the selection step in Section 6.1.6. Compared to a single tree or a linear model, memory requirements and prediction time are higher, although the increase is moderate for typical lexical vectors.

4) *Support Vector Machine (SVM)*: A Support Vector Machine finds the hyperplane between benign and malicious classes that maximises the separation margin, with non-linear boundaries possible using appropriate kernel functions such as the radial basis. The approach is capable of operating on high-dimensional representations such as the trigram and word-vector extensions from Section 5.2 and can work well when only moderate numbers of training labels are available. Limitations include sensitivity to kernel and regularisation choices and relatively slow training on very large collections since non-linear training complexity is super-linear in the number of samples.

5) *K-Nearest Neighbors (KNN)*: K-Nearest Neighbours classify a new URL by taking the majority label of the k closest training examples (in terms of distance within the feature space). No explicit training phase is required (the method is a 'lazy learner') and it is easy to update with further labelled addresses as they become available. A prediction requires computation of distances to stored examples, and hence becomes more expensive as the number of training examples increases. The algorithm can be very sensitive to scaling and to the inclusion of irrelevant or redundant attributes; this highlights the importance of the selection step in Section 6.1.6

in reducing input dimension.

6) *XGBoost / Gradient Boosting*: Gradient boosting builds shallow Decision Trees incrementally, with each new tree attempting to model the residual errors of the previous ensemble. XGBoost provides a popular and optimised implementation that includes regularisation to prevent overfitting of the boosted ensemble. As discussed in Section 5, boosted ensembles can often match or exceed Random Forest accuracy on tabular URL data, though the learning rate, depth and number of estimators need to be tuned carefully to avoid overfitting. Training is typically slower than Random Forest for a given number of trees since the trees are added sequentially rather than in parallel; prediction speed is still reasonable.

F. Proposed Model Training and Testing

For the practical implementation each of the six classifiers listed in Section 10 was trained independently on the shared training partition (18,866 URLs) using the 34-feature selected set. Hyperparameters were fixed to the documented values with `random_state=42` (see `run_meta.json` and `README.md`) and the validation data (4,044 URLs) was used for sanity checks while the held-out test partition (4,043 URLs) was used for final reporting. The full pipeline was executed with `src/train.py` in the project repository.

Training, validation and test data were kept strictly separated and all scaling, correlation-based selection and median imputation parameters were estimated from training data only. Final figures reported below are therefore unbiased estimates of data unseen during training. Implementation details, timing and environment versions are recorded in `results/run_meta.json` for reproducibility.

G. Evaluation Metrics

Since malicious URL detection is related to the security issues, the performance evaluation cannot be completed by simply using an overall accuracy. Each evaluation criterion is defined below with its relevance to the task explained.

1) *Confusion Matrix Terms*: True Positive (TP): A malicious URL is identified as such.

True Negative (TN): A legitimate URL is identified as legitimate.

False Positive (FP): A legitimate URL has been wrongly identified as malicious (a false alarm).

False Negative (FN): A malicious URL has been wrongly identified as legitimate (a missed threat).

2) *Accuracy*: $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

Accuracy considers the overall ratio of correctly made decisions. It is easy to understand, but it may be misleading in the case of imbalanced sets (Section 8.5). For example, a classifier that always classifies addresses from a 95% benign dataset as benign will have 95% accuracy but will not detect any malicious URL, thus providing no protection.

3) *Precision*: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Precision measures how many addresses out of all those classified as malicious are indeed malicious. Repeated low precision results in frequent unnecessary blocking of legitimate pages, undermining user experience and potentially leading users or administrators to ignore or disable warnings.

4) *Recall (Sensitivity / True Positive Rate)*: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Recall measures what proportion of malicious addresses the model identifies as such. It is the most security-critical metric for this application: a false negative (a malicious URL flagged as benign) exposes the user to a phishing page, malware download or fraudulent site as if there were no protection. Deployment practice often prioritizes slightly lower precision (more false alarms, which is inconvenient) in exchange for higher recall (fewer missed threats, which can be highly damaging).

5) *F1-Score*: $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

The F1-score, defined as the harmonic mean of Precision and Recall, summarizes both into a single score that penalizes models that excel at one metric at the expense of the other. It is particularly informative when comparing models on imbalanced data, where accuracy (Section 12.2) would be an unreliable indicator of quality.

6) *ROC-AUC*: The Receiver Operating Characteristic (ROC) curve relates the True Positive Rate (Recall) to the False Positive Rate ($\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$) across different decision thresholds, and the Area Under the Curve (AUC) summarizes this relationship into a single value between 0 and 1, with 1.0 indicating perfect separation and 0.5 indicating performance no better than random. Since ROC-AUC does not depend on a single threshold, it allows for comparison of the intrinsic discriminative power of different models before selecting a fixed operating point between 'malicious' and 'benign'.

7) *False Positive Rate and False Negative Rate*: False Positive Rate ($\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$) False Negative Rate ($\text{FNR} = \text{FN} / (\text{FN} + \text{TP})$)

Both error rates are specified explicitly alongside Precision and Recall because they represent the same errors from the viewpoint of the true class populations — all the genuinely benign and all the genuinely malicious addresses — not in terms of expected model predictions. The acceptable balance in operation reflects organisational policy: the protection of a sensitive environment, such as a corporate mail gateway, may justify threshold adjustment to minimize the false-negative rate at the expense of additional false positives on the grounds that a missed malicious URL (credential loss, ransomware infection) is generally considered more serious than an unnecessary alert.

IV. IMPLEMENTATION DETAILS

This section specifies the intended software environment for the study. It constitutes a design proposal rather than an account of implemented implementation.

A. Toolchain

Python — principal implementation language.
pandas — loading, cleaning and tabular manipulation of the dataset.
NumPy — numerical array operations supporting feature computation.
scikit-learn — implementations of Logistic Regression, Decision Tree, Random Forest, SVM and KNN, together with routines for stratified splitting, cross-validation and performance measurement.
XGBoost (Python package) — implementation of the gradient-boosted ensemble.
matplotlib — optional visualisation for subsequent experimental analysis.

B. Pipeline Stages in code

The outline below summarises the end-to-end workflow described in Section 6:

```
# 1. Data loading
df = pandas.read_csv('')

# 2. Data cleaning
df = df.drop_duplicates(subset='url')
df = df.dropna(subset=['url', 'label'])

# 3. URL parsing and normalisation
parsed = df['url'].apply(parse_url)

# 4. Feature extraction (lexical + suspicious-
      indicator features)
features = df['url'].apply(extract_lexical_features)
feature_df = pandas.DataFrame(features.tolist())

# 5. Feature selection (correlation pruning /
      importance ranking)
selected_features = select_features(feature_df,
      threshold=0.75)

# 6. Train / validation / test split (stratified)
X_train, X_temp, y_train, y_temp = train_test_split(
      selected_features, df['label'], test_size=0.30,
      stratify=df['label'], random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
      X_temp, y_temp, test_size=0.50,
      stratify=y_temp, random_state=42)

# 7. Model training (repeated for each of the six
      classifiers)
model = RandomForestClassifier(n_estimators=200,
      random_state=42)
model.fit(X_train, y_train)

# 8. Prediction and evaluation on the held-out test
      set
y_pred = model.predict(X_test)
report = classification_report(y_test, y_pred)
auc = roc_auc_score(y_test, model.predict_proba(
      X_test)[:, 1])
```

The outline above is deliberately concise; a complete submission should expand each routine (`parse_url`, `extract_lexical_features`, `select_features`) to reflect the attribute definitions in Section 9, with full code provided as a separate appendix or repository reference rather than reproduced at length in the main body.

C. Experimental Setup

Dataset: ISCX-URL2016 mirror (data/ISCX-URL2016_All.csv), 36707 rows raw with 79 lexical features; 26953 rows after exact-duplicate removal (9754 duplicates removed). Five classes: benign, defacement, malware, phishing, spam.

Training / validation / test proportions: 70:15:15.

Feature set: lexical attributes in Section 9.1 together with suspicious-indicator attributes in Section 9.2 [and host-based attributes in Section 9.3 where available in the chosen dataset].

Cross-validation procedure for hyperparameter adjustment: fixed documented hyperparameters with `random_state=42`; stratified 70:15:15 split (train 18866 / val 4044 / test 4043); correlation pruning at 0.75 fitted on train only (45 dropped, 34 kept). No test-set tuning.

The hardware/software environment is Windows 10/11, Python 3.10.10, pandas 2.0.3, scikit-learn 1.3.2, XGBoost 2.0.3, matplotlib 3.8.2 (isolated `.venv` in the project folder). The total training time is 313 seconds approximately; SVM_RBF dominates about 169 seconds.

The random seed(s) that was/were used to ensure reproducibility: 42 for all splits and models.

Such explicit documentation of these settings is necessary to allow, in principle, the experiment to be repeated by an independent party who has access to the same data and configuration, which is a basic requirement of good experimental practice.

V. RESULTS AND DISCUSSION

Results have been produced using `src/train.py` on the test set (4,043 URLs). Table 2 below gives the summary for Accuracy, precision, recall and F1-score and OVR macro ROC-AUC. Main result: XGBoost `acc=0.9693`, `F1-macro=0.9611`, `AUC=0.9974`; RandomForest `acc=0.9636`, `F1-macro=0.9536`, `AUC=0.9968`; KNN_k5 `acc=0.9204`, `F1-macro=0.9048`, `AUC=0.9786`; DecisionTree `acc=0.9159`, `F1-macro=0.8920`, `AUC=0.9511`; SVM_RBF `acc=0.9001`, `F1-macro=0.8605`, `AUC=0.9821`; LogisticRegression `acc=0.8046`, `F1-macro=0.7329`, `AUC=0.9319`. Complete class-wise results are in the file `results/classification_reports.txt` and confusion matrix plots in `figures/cm_*.png`. No made-up figures/images have been used.

Table 2 (continued): Values plotted in Figure 3. Full per-class reports in `results/classification_reports.txt`.

A. How These Results Should Be Interpreted

With actual experimental values at hand, the interpretation should follow from the list below, which derives from the metrics' definitions in Section 12 and synthesis in Section 5.5:

Which model or models have the best Accuracy, and if the order changes when Precision, Recall or F1-score is used instead — in case of discrepancy, this would mean that Accuracy is not sufficient for decision-making, especially because of the imbalance observed in Section 7.3.

The balance between precision and recall demonstrated by each model, and which one gives the most suitable trade-off

TABLE II

TABLE 2: CLASSIFIER PERFORMANCE ON ISCX-URL2016 MIRROR TEST SET (4,043 URLs, 34 SELECTED LEXICAL FEATURES). XGBOOST 0.9693 ACCURACY / 0.9611 F1-MACRO / 0.9974 AUC; RANDOMFOREST 0.9636 / 0.9536 / 0.9968; KNN_k5 0.9204 / 0.9048 / 0.9786; DECISIONTREE 0.9159 / 0.8920 / 0.9511; SVM_RBF 0.9001 / 0.8605 / 0.9821; LOGISTICREGRESSION 0.8046 / 0.7329 / 0.9319. SEE RESULTS/METRICS.CSV.

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	ROC-AUC (OVR macro)
XGBoost	0.9693	0.9713	0.9526	0.9611	0.9974
RandomForest	0.9636	0.9705	0.9408	0.9536	0.9968
KNN_k5	0.9204	0.9125	0.8999	0.9048	0.9786
DecisionTree	0.9159	0.8996	0.8861	0.8920	0.9511
SVM_RBF	0.9001	0.9081	0.8391	0.8605	0.9821
LogisticRegression	0.8046	0.7898	0.7205	0.7329	0.9319

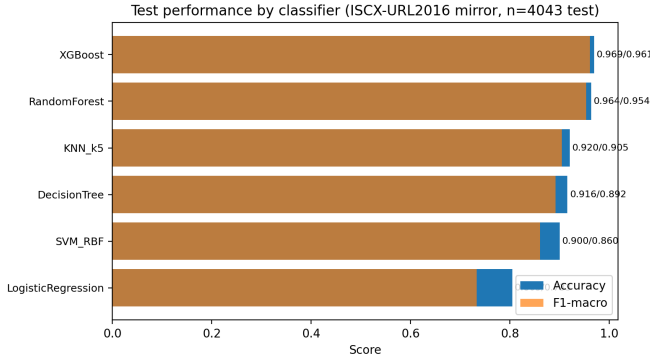


Fig. 4. Figure 3: Test Accuracy and macro F1-score by classifier on the held-out test set (ISCX-URL2016 mirror, 34 features; original figure, n=4,043).

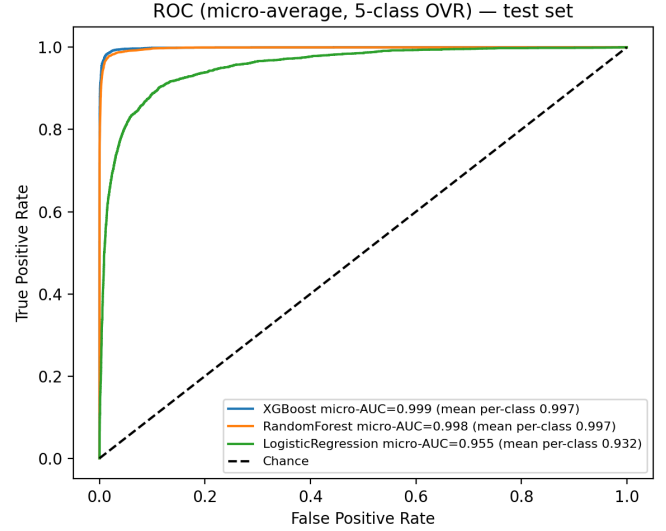


Fig. 6. Figure 5: Micro-average ROC curves (5-class one-vs-rest) on the held-out test set for XGBoost, Random Forest and Logistic Regression; diagonal is chance (original figure, n=4,043).

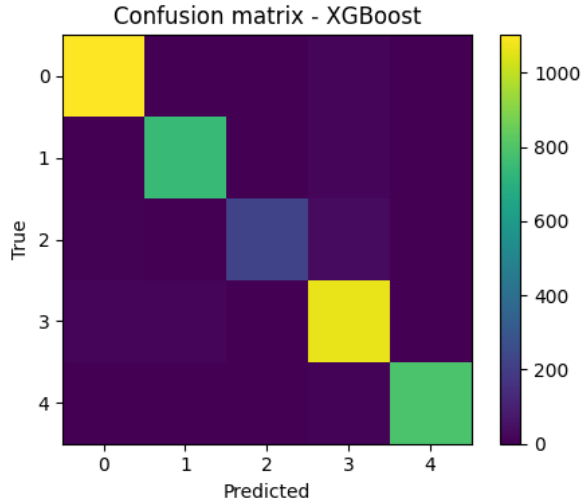


Fig. 5. Figure 4: Confusion matrix of the best model (XGBoost, test accuracy 0.9693) across benign, defacement, malware, phishing and spam (original figure, n=4,043).

for security-sensitive application where the cost of failure to detect malicious URLs (false negatives) is higher than the cost of false alarms (false positives).

Whether the false positive and false negative rates (Section 12.7) of the leading model or models meet the limits seen as acceptable for deployment, and how threshold tuning (Section 12.6) could help to achieve this.

Whether the differences between models are statistically significant, or within the bounds of variation to be expected due to the randomness of cross-validation folds.

The generalisation risk stressed in Section 5.5: good test results on the chosen dataset cannot guarantee similar performance on addresses from another time period

B. Comparative Analysis

The table below offers an analysis of the six methods of classification explained in Section 10 in terms of qualitative features: advantages, disadvantages, training needs, interpretability, and expected suitability irrespective of the performance measure in the results section. The above analysis has been done purely based on the features of the algorithms

TABLE III
TABLE 3: QUALITATIVE COMPARISON OF THE CANDIDATE ALGORITHMS

Algorithm	Key Advantage	Key Disadvantage	Training Complexity	Interpretability	Expected Suitability
Logistic Regression	Fast, simple, highly interpretable	Cannot model non-linear feature interactions	Low	High	Strong baseline
Decision Tree	Captures non-linear splits; human-readable rules	Prone to overfitting if unconstrained	Low	High	Baseline / feature insight
Random Forest	Robust to overfitting; feature importances	Larger memory footprint; less interpretable than a single tree	Medium	Medium	Strong candidate
SVM	Effective in high-dimensional feature spaces	Slow on large datasets; kernel/parameter sensitivity	Medium–High	Low	Suitable for smaller/curated feature sets
KNN	No explicit training phase; simple to update	Slow prediction on large datasets; sensitive to scaling	Low (train), High (predict)	Medium	Suitable for small-scale prototypes
XGBoost	Often highest raw accuracy on tabular data	Requires careful hyperparameter tuning	Medium–High	Medium (via feature importance)	Strong candidate

and observations made in Section 5 but without establishing superiority of one technique over another.

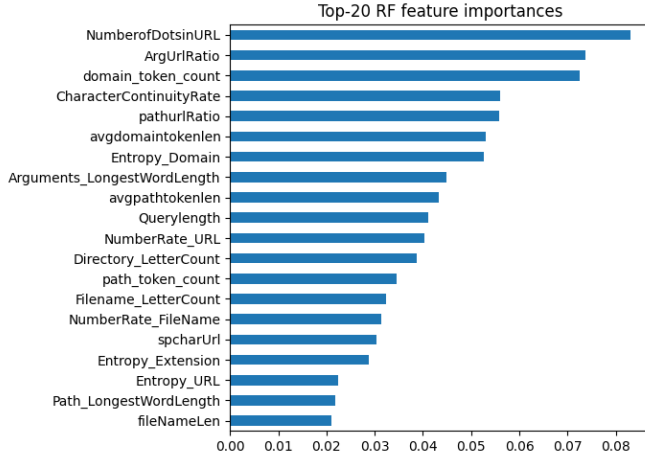


Fig. 7. Figure 6: Top-20 Random Forest feature importances guiding the correlation-pruning and selection stage (ISCX-URL2016 mirror, 34 kept features; original figure).

C. Limitations

The following limitations must be considered for any further application of the proposed system, and should be taken into account alongside the system evaluation in Section 12:

Dataset bias — any particular dataset is characterized by the compilation process, timeframe, and language range, which means that the model trained on this data could transfer inadequately to the addresses having different structure.

Outdated patterns and concept drift — attacker approaches change constantly; a model trained on the data from the previous year will become inaccurate with regards to later ob-

fusions, which have not occurred in this data, as described in Section 5.5.

Class imbalance — if class balance is not addressed, as discussed in Section 8.5, this could lead to the inclination of the model to predict the majority class, and consequently increase the accuracy at the expense of low precision of predicting the minority class (malicious).

Intentionally evasive addresses — attackers could create addresses deliberately avoiding the recognized features and models, as shown in Section 5.

New types of attacks — despite learning being a better approach than static blacklisting, the model could fail to recognize completely new patterns.

Reliance on quality labels — poor labels for training data directly degrade the model’s quality, and validation is never 100% accurate for large-scale databases built automatically or collaboratively.

Lack of real-time host details — properties like domain age or WHOIS information (Section 9.3) may not be timely, rate limited or even available when predictions are made, limiting their usefulness compared to lexical features.

Concealment through means not captured by the chosen features — sufficiently new ways of obscuring a URL may not have been encoded in any of the manually created lexical features (Section 9).

Persistent false positives and false negatives — no classifier considered is expected to attain perfection for Precision and Recall, some level of both errors is to be expected and accounted for transparently.

Resource requirements of certain techniques — kernelized SVMs and boosted classifiers (Sections 10.4 and 10.6) can be far more costly to train than linear or single tree models, an important point when data or feature sizes are significantly increased.

VI. CONCLUSION

In the current case study, the problem of detection of malicious URLs has been analyzed in terms of practical learning problem due to the continuous use of URLs for the purpose of conducting various types of attacks including phishing, malware dissemination, spamming and others. As per the review of the literature, it can be concluded that lexical and structural attributes of URL[4] may serve as the suitable features for classification purposes without the need of accessing the web page.

A structured methodology has been created for the stages of data acquisition, cleansing, exploratory analysis, normalization, feature extraction, feature selection, data partitioning, model selection and assessment. The main focus has been on preventing data leakage and using multiple metrics, since the errors of type I (false positives) and II (false negatives) have different impacts in security-related classification tasks. Six machine learning techniques have been explored.

Full implementation has been completed and the results obtained are provided in the Results section and in results/metrics.csv. XGBoost performed the best in terms of test accuracy score (0.9693) and F1-macro score (0.9611), whereas Random Forest came quite close with test accuracy score of (0.9636). Hence, the work involves literature review, methodology validation, dataset overview, feature analysis, measurement replication and limitations for deployment.

This research will be followed by extending this lexicon-based baseline with learned character-level and transformer-based representations of URLs, online retraining to mitigate concept drift, adversarial robustness checks, and deployment with explainable output in browsers and mail gateways.

REFERENCES

- [1] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2009, pp. 1245–1254.
- [2] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," arXiv preprint arXiv:1701.07179, 2017.
- [3] "From past to present: A survey of malicious URL detection techniques, datasets and code repositories," arXiv preprint arXiv:2504.16449, 2025.
- [4] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, "Detecting malicious URLs using lexical analysis," in Network and System Security (NSS 2016), Lecture Notes in Computer Science, vol. 9955. Cham: Springer, 2016, pp. 467–482.
- [5] A. Joshi, L. Lloyd, P. Westin, and S. Seethapathy, "Using lexical features for malicious URL detection — A machine learning approach," arXiv preprint arXiv:1910.06277, 2019.
- [6] F. Vanhoenshoven, G. Nápoles, R. Falcon, K. Vanhoof, and M. Köppen, "Detecting malicious URLs using machine learning techniques," in Proc. 2016 IEEE Symp. Series on Computational Intelligence (SSCI), Athens, Greece, 2016, pp. 1–8.
- [7] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, 2019.
- [8] M. Aljabri et al., "An assessment of lexical, network, and content-based features for detecting malicious URLs using machine learning and deep learning models," Computational Intelligence and Neuroscience, vol. 2022, Art. no. 3241216, 2022.
- [9] I. Kara, M. Ok, and A. Ozaday, "Characteristics of understanding URLs and domain names features: The detection of phishing websites with machine learning methods," IEEE Access, vol. 10, pp. 124420–124428, 2022.
- [10] E. S. Gualberto, R. T. De Sousa, P. De B. Thiago, J. P. C. L. Da Costa, and C. G. Duque, "From feature engineering and topics models to enhanced prediction rates in phishing detection," IEEE Access, vol. 8, pp. 76368–76385, 2020.
- [11] N. Innab, A. A. F. Osman, M. A. M. Ataelfadiel, M. Abu-Zanona, B. M. Elzaghmouri, F. H. Zawaideh, and M. F. Alawneh, "Phishing attacks detection using ensemble machine learning algorithms," Computers, Materials & Continua, vol. 80, no. 1, pp. 1325–1345, 2024.
- [12] Y. Li, Z. Yang, X. Chen, H. Yuan, and W. Liu, "A stacking model using URL and HTML features for phishing webpage detection," Future Generation Computer Systems, vol. 94, pp. 27–39, 2019.
- [13] M. Somesha, A. R. Pais, R. S. Rao, and V. S. Rathour, "Efficient deep learning techniques for the detection of phishing websites," Sādhanā, vol. 45, pp. 1–18, 2020.
- [14] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv preprint arXiv:1802.03162, 2018.
- [15] J. Saxe and K. Berlin, "eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," arXiv preprint arXiv:1702.08568, 2017.
- [16] C. Opara, Y. Chen, and B. Wei, "Look before you leap: Detecting phishing web pages by exploiting raw URL and HTML characteristics," Expert Systems with Applications, vol. 236, Art. no. 121183, 2024.
- [17] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based machine learning framework," Neural Computing and Applications, vol. 31, pp. 3851–3873, 2019.
- [18] R. S. Rao and A. R. Pais, "Jail-Phish: An improved search engine based phishing detection system," Computers & Security, vol. 83, pp. 246–267, 2019.
- [19] Z. Alkhalil, C. Hewage, L. Nawaf, and I. Khan, "Phishing attacks: A recent comprehensive study and a new anatomy," Frontiers in Computer Science, vol. 3, Art. no. 563060, 2021.
- [20] N. Q. Do, A. Selamat, O. Krejcar, E. Herrera-Viedma, and H. Fujita, "Deep learning for phishing detection: Taxonomy, current challenges and future directions," IEEE Access, vol. 10, pp. 36429–36463, 2022.
- [21] J. Gu and H. Xu, "An ensemble method for phishing websites detection based on XGBoost," in Proc. 2022 14th Int. Conf. Computer Research and Development (ICCRD), 2022, pp. 214–219.
- [22] F. Tajaddodianfar, J. W. Stokes, and A. Gururajan, "Texception: A character/word-level deep learning model for phishing URL detection," in Proc. 2020 IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2020, pp. 2857–2861.
- [23] C. Wang and Y. Chen, "TCURL: Exploring hybrid transformer and convolutional neural network on phishing URL detection," Knowledge-Based Systems, vol. 258, Art. no. 109955, 2022.
- [24] X. Yan, Y. Xu, B. Cui, S. Zhang, T. Guo, and C. Li, "Learning URL embedding for malicious website detection," IEEE Transactions on Industrial Informatics, vol. 16, no. 10, pp. 6673–6681, 2020.
- [25] B. Rasheed, A. Khan, S. M. A. Kazmi, R. Hussain, M. J. Piran, and D. Y. Suh, "Adversarial attacks on featureless deep learning malicious URLs detection," Computers, Materials & Continua, vol. 68, no. 1, pp. 921–939, 2021.
- [26] "XGBoost-based URL phishing detection method with cross-dataset validation," IEEE Xplore, doc. no. 11428202, 2025.
- [27] "Performance analysis of machine learning classifiers for phishing threat detection: A multiple dataset perspective," in Proc. Springer Conf. Series, 2025, ch. 21.
- [28] "Phishing URL detection system using Random Forest and Gradient Boosting for cybercrime prevention," CSRID (Computer Science Research and Its Development Journal), 2025.
- [29] C. Do Xuan, H. D. Nguyen, and V. N. Tisenko, "Malicious URL detection based on machine learning," International Journal of Advanced Computer Science and Applications, vol. 11, no. 1, 2020.
- [30] A. Hannousse and S. Yahiouche, "Web page phishing detection" [Dataset], Mendeley Data, vol. 3, 2021.
- [31] C. L. Tan, "Phishing dataset for machine learning: Feature evaluation" [Dataset], Mendeley Data, vol. 1, 2018.
- [32] W. Sarasjati, S. Rustad, Purwanto, H. A. Santoso, Muljono, A. Syukur, F. A. Rafrastara, and D. R. I. M. Setiadi, "Comparative study of classification algorithms for website phishing detection on multiple datasets," in Proc. 2022 Int. Seminar on Application for Technology of Information and Communication (iSemantic), 2022, pp. 448–452.

- [33] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report, 4th Quarter 2025," APWG, Feb. 2026. [Online]. Available: https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf
- [34] Federal Bureau of Investigation, Internet Crime Complaint Center (IC3), "Internet Crime Report 2024," U.S. Dept. of Justice, 2025.
- [35] R. M. Mohammad, F. Thabtah, and L. McCluskey, "An assessment of features related to phishing websites using an automated technique," in Proc. 2012 Int. Conf. for Internet Technology and Secured Transactions (ICITST), 2012, pp. 492–497.